Print / Save as PDF

Nexus Fleet Control

System Architecture & Capabilities Report (v1.0.0)

1. Executive Summary

The **Nexus Fleet Control** system is a cloud-based robotics management platform designed to monitor, command, and coordinate a fleet of autonomous ground vehicles (AGVs) or mobile robots. It bridges the gap between hardware-level robotics systems (ROS) and a user-friendly, real-time web interface.

This report details the underlying technology stack, the bidirectional data flow architecture, the comprehensive suite of integrated sensors, and the provisioning process for connecting physical hardware to the platform.

2. Technology Stack

Frontend (Client Interface)

- **Framework:** React.js (built with Vite for highly optimized HMR and bundling).
- **Styling & UI:** Custom CSS3 implementing a modern "Glassmorphism" aesthetic with a Dark Mode primary theme.
- **State Management:** React Context API (`AppContext`) for global fleet state and websocket references.
- **Mapping Engine:** HTML5 Canvas and SVG overlays for dynamic, smooth rendering of floor plans, waypoints, and live robot poses.
- **Internationalization:** `react-i18next` for multi-language support.

Backend (Cloud Server)

- **Runtime:** Node.js with strict TypeScript compilation.
- **Web Server:** Express.js handling REST API endpoints, static assets, and middleware.
- **Real-Time Communication:** `ws` (WebSockets) running a custom pub/sub broker on Port 5000.
- **Database:** PostgreSQL (hosted via Supabase), managed using the native `pg` connection pool.
- **Security:** JWT validation (JSON Web Key Sets), `bcryptjs` for API Key hashing, and `helmet` for HTTP header security.

Robotics Middleware

- **Framework:** Robot Operating System (ROS 1 or ROS 2).
- **Cloud Integration:** `rosbridge_suite` which provides a standard JSON API to the ROS network over WebSockets.

3. Real-Time Data Flow Architecture

The system operates on an event-driven, bidirectional WebSocket architecture to ensure sub-100ms latency between physical robot movement and UI updates.

```
[ PHYSICAL ROBOT ]                                  [ CLOUD BACKEND ]
                          [ WEB DASHBOARD ]
 (ROS Navigation)                                     (Node.js / WS)
                                (React)
            |                                                |
                                  |
            |--- 1. ROS publishes Odometry (/odom) ----------->|
                                  |
 |                                                |--- 2. Backend parses &
                     packages as JSON ---->    |
            |                                                |
                   |--- 3. React updates map Canvas
            |                                                |
                                  |
            |<-- 6. ROS translates to motor voltage (/cmd) --- |
                                  |
|                                                |<-- 5. Backend validates &
                       formats to Twist --    |
            |                                                |
                |--- 4. User clicks joystick (Teleop)
```

The Uplink (Telemetry Pipeline)

Robots are configured to expose a WebSocket port via `rosbridge_server`. The backend Node.js server acts as a *client*, dialing into the robot and subscribing to critical topics (like `/odom` and `/scan`). When the robot moves, it publishes hardware data. The backend intercepts this, normalizes the payload, attaches the `robotId`, and broadcasts a centralized JSON packet to all currently logged-in React clients.

The Downlink (Command Pipeline)

When a human operator interacts with the dashboard (e.g., clicks the "Emergency Stop" button or draws a route), the React application sends a JSON command to the Node.js backend. The backend verifies the user's IAM permissions against the database. If authorized, the backend translates the web command into a standardized ROS message (e.g., a

`geometry_msgs/Twist` for manual driving or `geometry_msgs/PoseStamped` for autonomous routing) and injects it directly into the robot's hardware bus.

4. Integrated Sensors & Implementations

The dashboard is pre-configured to ingest, process, and visualize data from several standard robotics sensors.

1. 2D LIDAR (Light Detection and Ranging)

ROS Topic: /scan **Type: sensor_msgs/LaserScan**

Hardware: Typically a spinning laser mounted on top of the robot (e.g., RPLIDAR, Sick, Hokuyo) that measures distance to obstacles in a 360-degree plane.

Dashboard Implementation: The backend receives an array of distances (ranges) and angles. This is rushed to the frontend's `LidarViewer.jsx` component, which utilizes a highly optimized HTML5 Canvas to mathematicaly map polar coordinates to Cartesian coordinates, rendering a real-time red dot scatter plot over a radar grid. This allows operators to "see" what the robot sees, identifying walls, moving humans, or debris.

2. Wheel Encoders & IMU (Odometry)

ROS Topic: /odom **Type: nav_msgs/Odometry**

Hardware: Magnetic or optical encoders on the wheel motors tracking exact rotations, paired with an Inertial Measurement Unit (IMU) tracking acceleration and gyroscopic rotation.

Dashboard Implementation: The robot's Navigation stack fuses this data to determine its exact (X, Y) coordinate and Heading (Theta) on the map. The backend parses this quaternion math into a simple 2D heading. The frontend uses this to smoothly animate the robot's avatar across the uploaded floor plan in the `RouteEditor.jsx` canvas.

3. Battery Management System (BMS)

ROS Topic: /battery_state **Type: sensor_msgs/BatteryState**

Hardware: Voltage monitors connected directly to the robot's LiPo or SLA batteries.

Dashboard Implementation: A continuous stream of voltage and percentage data is forwarded to the Fleet Status sidebar, ensuring operators can preemptively dock robots before they lose power mid-mission.

4. Camera Array (Visual Feed)

ROS Topic: /image_raw/compressed **Type: sensor_msgs/CompressedImage**

Hardware: USB Webcams, Intel RealSense depth cameras, or Raspberry Pi Camera modules.

Dashboard Implementation: Due to bandwidth constraints, the dashboard is designed to render compressed JPEG streams (MJPEG). The `CameraFeed.jsx` component mounts an `<img>` tag that rapidly refreshes with Base64 payloads or binds to a dedicated WebRTC/MJPEG streaming proxy port on the robot, providing operators with a first-person view (FPV) during manual teleoperation.

5. System Diagnostics (CPU/Temp)

Hardware: Onboard computer (e.g., Raspberry Pi, Jetson Nano) internal thermal sensors.

Dashboard Implementation: The system monitors the core computational health of the robot to prevent thermal throttling or crashes during computationally expensive SLAM (Simultaneous Localization and Mapping) operations.

5. Hardware Integration Guide

Follow these steps to transition from the software simulator to a physical robot (e.g., a Raspberry Pi chassis).

Step 1: Prepare the Robot Hardware

1. Ensure the robot has a functioning onboard computer (Raspberry Pi/Jetson) running Ubuntu Linux.
2. Verify that a ROS environment (ROS Kinetic, Noetic, or ROS 2 Humble) is installed.
3. Ensure the robot's base controller is actively publishing to `/odom` and subscribing to `/cmd_vel`.

Step 2: Install the Communication Bridge

The dashboard requires `rosbridge_suite` to communicate with the robot. Install it via terminal on the robot:

```
# For ROS 1 (Noetic)
sudo apt-get install ros-noetic-rosbridge-server
roslaunch rosbridge_server rosbridge_websocket.launch port:=9090

# For ROS 2 (Humble)
sudo apt-get install ros-humble-rosbridge-suite
ros2 run rosbridge_server rosbridge_websocket --port 9090
```

Make a note of the robot's local IP address (e.g., `192.168.1.150`). The WebSocket URL will be `ws://192.168.1.150:9090`.

Step 3: Register the Robot in the Dashboard

1. Log into the Nexus Fleet Control dashboard using an Administrator account.
2. Access the underlying database configuration (or Robot Management UI if available).
3. Create a new robot record. Set the `name` and explicitly supply the `ros_bridge_url` you derived in the previous step.
4. Generate an API Key hash for secure authentication.

Step 4: Network Connectivity

Ensure that the cloud server hosting the Node.js backend can route traffic to the robot's IP address. If the robot is on a private network, you may need a VPN (like Tailscale or ZeroTier) so the cloud server can "reach in" and connect to the robot's port 9090.

Step 5: Initialization & Mapping

1. Start your Node.js backend. It will detect the registered robot, attempt a WebSocket handshake, and mark it **ONLINE**.
2. Use the physical robot to teleoperate around your facility while running a mapping node (e.g., `gmapping` or `slam_toolbox`).
3. Save the generated map as an image file (PNG/PGM).
4. In the dashboard's **Mission Planner**, click "Upload Map", provide the image file, and input the real-world dimensions to scale the Canvas correctly.
5. **Complete:** Your robot will now appear on the dashboard map in real-time. You can draw routes and issue commands immediately.

Generated by Antigravity AI • Nexus Fleet Control Cloud Subsystem